Data Replication in a Single-Leader Environment

CENG465 — Principles of Data-Intensive Systems
Progress Presentation

Mert Karahan Doğukan Topçu

CENG465

May 5, 2026

Outline

What Is This Project About?

Goal: Build and observe a single-leader replication system across two physical machines.

Required behavior:

- One node acts as Leader (Primary) — receives all writes
- One node acts as Follower (Secondary) — replicates from leader
- Measure when and how quickly writes propagate
- Log every operation with ordering metadata

Deployment: Two separate macOS machines on the same LAN — satisfies the project's *truly distributed* requirement.

Project Scope (Progress)

1. **Environment Setup**
   Replica set on two machines
2. **Schema Design**
   Replication-aware data model
3. **Replication Logging**
   Operation logs with timestamps
4. **Insert / Update / Delete**
   Visibility and delay tracking

Database Choice: MongoDB Replica Set

Key requirements from the project:

- Single-leader replication
- Observable update propagation
- Support for insert, update, delete
- Replication lag must be measurable

MongoDB Replica Set provides all of this natively:

- Primary node accepts writes
- Secondary applies operations from the **oplog**
- Replication is asynchronous — lag is observable
- `rs.status()` exposes health and lag in real time

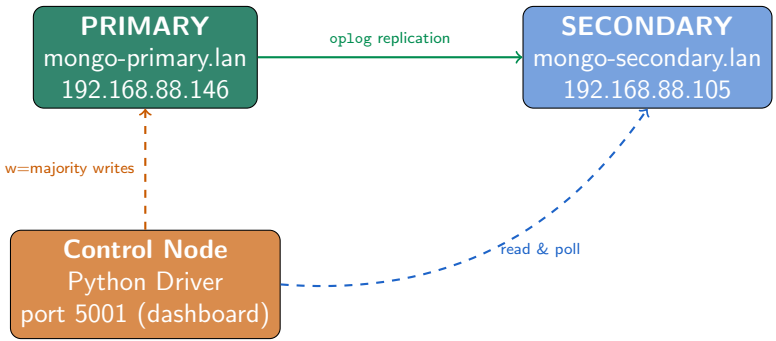
Raft-Inspired Metadata

Rather than implementing full Raft, we embed Raft-style tracking fields directly in our documents:

- `leader_term`
- `log_index`
- `operation_id`
- `version`

This allows analysis of **update ordering** and **visibility** without replacing MongoDB's built-in replication.

System Architecture



Replica set name: `rs0` **Port:** 27017

Write concern: `w="majority"` — primary confirms before returning

Installation Challenges

Challenge 1: Duplicate `bindIp` in `mongod.conf`

The Homebrew default config and our LAN IP entry were both present under `net:`, causing `mongod` to fail on startup silently.

Fix: Removed duplicate key, kept only `bindIp: 127.0.0.1,192.168.88.146`

Challenge 2: Non-Breaking Spaces in Config File

Copying YAML from a chat message introduced Unicode non-breaking spaces (`0xC2A0`) instead of ASCII spaces. `mongod` reported `"Unrecognized option: systemLog"`.

Fix: Rewritten with `cat > mongod.conf << 'EOF'` to guarantee ASCII encoding.

Challenge 3: `sudo brew services` Ownership Issue

Running `sudo brew services` changed file ownership to root, preventing the service from starting as a user process.

Fix: `sudo chown -R $(whoami)` on affected paths, then restarted without sudo.

Installation Challenges (cont.)

Challenge 4: Missing Log and Data Directories

`mongod` failed silently because `/opt/homebrew/var/log/mongodb/` did not exist yet.
Fix: `mkdir -p` for both log and data directories before starting the service.

Challenge 5: macOS AirPlay Receiver on Port 5000

Flask defaulted to port 5000, which is reserved by macOS AirPlay Receiver (ControlCenter process).
Fix: Moved Flask dashboard to port `5001`.

Challenge 6: Secondary Not Reachable from Primary

Secondary Mac's `mongod` was binding only to `127.0.0.1` and not the LAN IP, making port 27017 unreachable from the primary.
Fix: Corrected `bindIp` to include `192.168.88.105`, recreated missing directories, restarted service.

Schema: `items` Collection

Stores application data with replication-aware tracking fields.

```
{
  "_id":               ObjectId,
  "key":               string,          # human-readable identifier
  "value":             object,          # application payload
  "version":           int,             # increments on every write
  "leader_term":       int,             # Raft-inspired term number
  "last_log_index":    int,             # global operation order
  "last_operation_id": UUID string,     # unique per write
  "last_updated":      ISODate,         # timestamp of last write on leader
  "deleted":           bool,            # soft-delete flag
  "created_by":        string           # client identifier
}
```

Why `version`?
Allows the control node to poll the secondary and detect *exactly* when a specific write becomes visible — without relying on timestamps alone.

Why soft delete?
Preserves the document on secondary for replication analysis. Hard deletes would remove the evidence of propagation timing.

Schema: operation_logs Collection

Every write produces one log entry. This collection is the primary source of evidence for replication behavior.

```
{
  "operation_id":           UUID string,
  "leader_term":            int,
  "log_index":              int,                    # global ordering
  "operation_type":         "insert" | "update" | "delete",
  "target_id":              ObjectId,
  "leader_write_time":      ISODate,                # write confirmed on primary
  "follower_visible_time":  ISODate | null,         # secondary returned correct version
  "replication_delay_ms":   float | null,           # follower_visible - leader_write
  "version_before":         int | null,
  "version_after":          int,
  "status":                 "visible_on_follower" | "timeout"
}
```

Key Insight

`replication_delay_ms = follower_visible_time - leader_write_time`

This is measured by polling the secondary every 10ms until `version == version_after`.

Implementation Overview

Control Node (Python)

- **config.py** — hosts, ports, timeouts
- **db.py** — primary & secondary connections
- **operations.py** — insert/update/delete with logging
- **app.py** — Flask web dashboard (port 5001)
- **benchmark.py** — 1000-op stress test
- **test_replication.py** — 13 assertions
- **trace.py** — live change stream watcher

Write Flow (every operation)

1. Write to primary with `w="majority"`
2. Record `leader_write_time`
3. Poll secondary every 10ms
4. When `version` matches: record `follower_visible_time`
5. Compute `replication_delay_ms`
6. Insert into `operation_logs`

Timeout: 5000ms — if secondary does not show the update within 5 seconds, the log entry status is set to `"timeout"`.

Replica Set Verification

After both machines were running, the replica set was initialized from the primary:

`rs.initiate()` **command:**

- `_id: "rs0"`
- Member 0: `mongo-primary.lan:27017` (priority 2)
- Member 1: `mongo-secondary.lan:27017` (priority 1)

Verified result

`mongo-primary.lan:27017` PRIMARY
`mongo-secondary.lan:27017` SECONDARY

Write concern: `w="majority"`

Every write operation uses `WriteConcern(w="majority")`, which means MongoDB waits for the primary *and* at least one secondary to acknowledge the write before confirming to the client.

Test Suite: 13/13 Passed

5 test categories:

1. **Basic CRUD** — insert, update, soft delete visible on secondary
2. **Burst writes** — 10 rapid inserts, all replicated
3. **Large payload** — 50KB document, data integrity check
4. **Sequential versions** — 5 updates, correct final version on secondary
5. **Monotonic versions** — secondary never returns a stale version

Result: 13/13 PASS

- Success rate: 100%
- 0 timeouts across all test cases
- Secondary always reflected correct version
- Soft delete flag propagated correctly
- Version monotonicity held across all reads

Benchmark: 1000 Operations

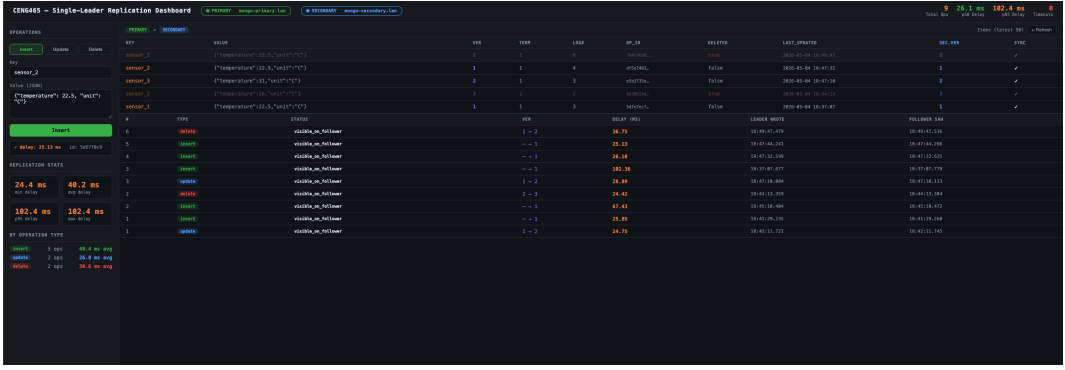
Operation mix:

- 417 inserts (41.7%)
- 377 updates (37.7%)
- 206 deletes (20.6%)

Replication delay:

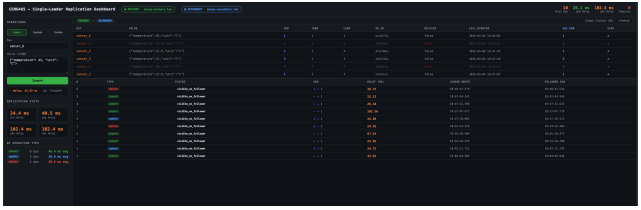
Metric	Value
min	4.56 ms
p50	7.70 ms
p95	12.78 ms
p99	96.40 ms
max	113.52 ms
avg	10.17 ms

Web Dashboard — Overview



Live dashboard at `http://192.168.88.146:5001` — accessible from both machines simultaneously.

Web Dashboard — Insert Operation

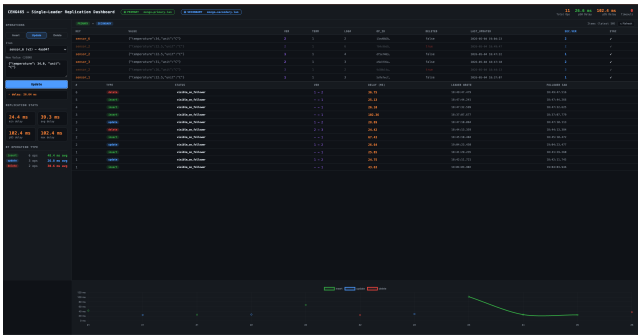


Insert flow:

1. User enters key + JSON value
2. Write sent to PRIMARY with `w=majority`
3. Control node polls SECONDARY every 10ms
4. When `version=1` appears on secondary: delay recorded
5. Log entry written to `operation_logs`

Result appears instantly in the table with `leader_write_time` and `follower_visible_time`.

Web Dashboard — Update Operation

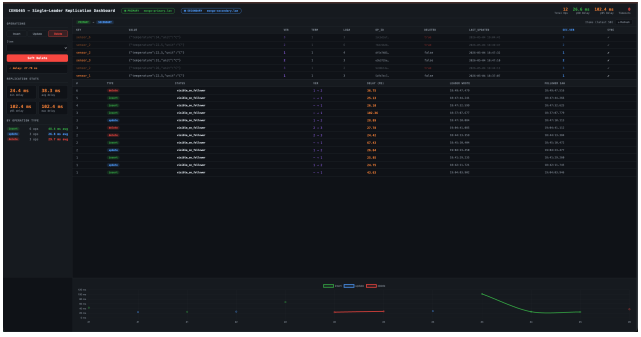


Update flow:

1. Select existing item from dropdown
2. New JSON value sent to primary
3. `version` increments (e.g. 1 → 2)
4. Secondary polled until `version` matches
5. `replication_delay_ms` computed and logged

The **SEC.VER** column in the table shows the secondary's current version — a mismatch indicates replication in progress.

Web Dashboard — Delete Operation



Soft Delete flow:

1. Select item to delete
2. `deleted: true` written to primary
3. `version` still increments
4. Document remains on secondary — only flag changes
5. Replication delay measured identically

Soft delete is chosen over hard delete to preserve the document on the secondary for replication timing analysis.

Conclusion & Next Steps

Completed (Progress Presentation):

- ✓ MongoDB Replica Set on 2 physical machines
- ✓ Replication-aware schema with version tracking
- ✓ Insert / Update / Soft Delete operations
- ✓ Replication delay measurement & logging
- ✓ 1000-op benchmark (100% success)
- ✓ 13/13 test assertions passing
- ✓ Live web dashboard (cross-machine)

Planned (Final Report):

- Eventual consistency experiment
- Monotonic reads experiment
- Read-after-write consistency
- Concurrent writes analysis
- Export logs as CSV/JSON
- Statistical analysis & graphs
- Final report & presentation PDF

Thank you.

Questions?